CIS5200: Machine Learning Spring 2026

4: Binary Classification with Linear Predictors

Date: January 27, 2026 *Surbhi Goel*

Acknowledgements. These notes are heavily inspired by Chapter 9 of Understanding Machine Learning: From Theory to Algorithms (UML) and Cornell University's CS 4/5780 — Spring 2022. Includes edits from Jake Gardner.

Disclaimer. These notes have not been subjected to the usual scrutiny reserved for formal publications. If you notice any typos or errors, please reach out to the author.

1 Linear Predictors

A linear predictor makes its decisions by computing a weighted sum of input features. For binary classification (where $\mathcal{Y} = \{-1, 1\}$), it predicts based on whether this sum is positive or negative. While this may seem restrictive, linear predictors are remarkably powerful and serve as building blocks for more complex models we'll study later.

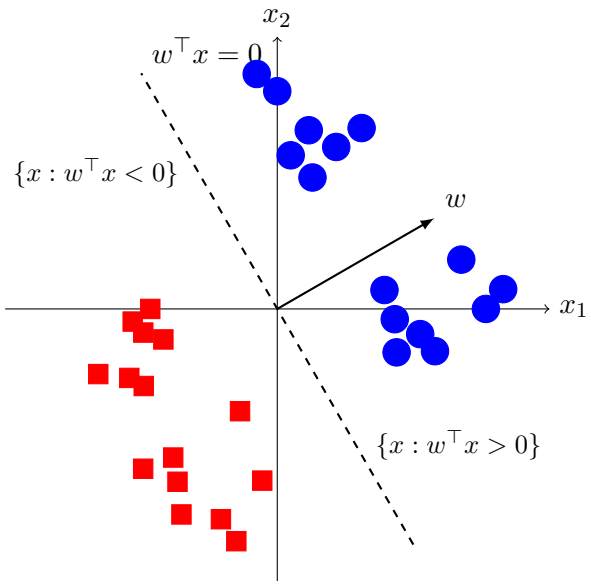


Figure 1: A linear predictor divides the space into two regions using a hyperplane $w^\top x = 0$.

Examples. Let's revisit two examples from our first lecture to build intuition:

1. **House Price Prediction (Regression):** A linear predictor estimates the price by computing:
$$\text{price} = w_1 \times \text{square footage} + w_2 \times \text{num bedrooms} + w_3 \times \text{age} + \cdots$$

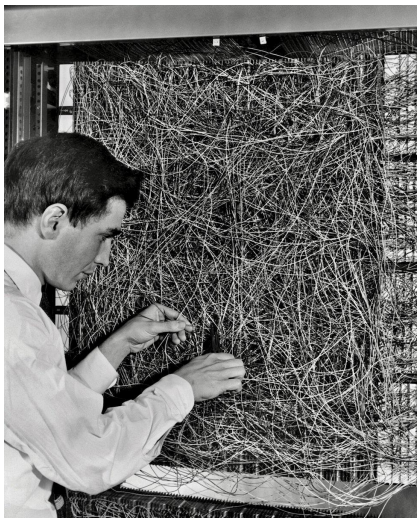
The weights $(w_1, w_2, \ldots)$ tell us how important each feature is - a larger weight for square footage would mean this feature strongly influences the predicted price.

2. **Spam Detection (Binary Classification):** For classifying emails, a linear predictor computes:
$$\text{score} = w_1 \times \text{freq}(\text{“money”}) + w_2 \times \text{freq}(\text{“urgent”}) + \cdots$$
If this score is positive, it predicts +1 (spam); if negative, it predicts -1 (not spam).

These examples show how linear predictors, despite their simplicity, can capture meaningful patterns in data through careful weighting of features. The key question is: *how do we find these weights?*

In this lecture, we'll study one of the earliest and most influential algorithms for learning these weights: the Perceptron algorithm. Developed by Frank Rosenblatt in 1957, it was one of the first machine learning algorithms that could automatically learn from data.

The algorithm gained significant attention for its simplicity and biological motivation, with the New York Times in 1958 famously declaring it an “Electronic Brain That Teaches Itself” (NYT, 1958). While modern deep learning has far surpassed the Perceptron's capabilities, understanding its properties provides crucial insights into why learning algorithms work. Let's now formalize this mathematically.



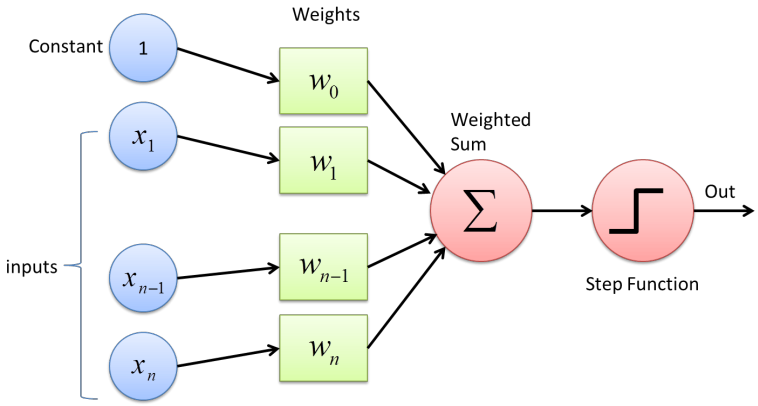


Figure 2: Frank Rosenblatt with the Mark I Perceptron (1960) and its schematic diagram.

2 Binary Classification with Linear Predictors

While we saw both regression and classification examples earlier, we'll focus on binary classification for the rest of this lecture. Let us formalize the setup we will use.

- Input space $\mathcal{X} = \mathbb{R}^d$ (each input x is a d-dimensional vector of features, like word frequencies in our spam example)

- Label space $\mathcal{Y} = \{-1, 1\}$[1]
- Hypothesis class $\mathcal{H} := \{x \mapsto \text{sgn}(w^\top x + b) \mid w \in \mathbb{R}^d, b \in \mathbb{R}\}$ where $\text{sgn}(a) = \begin{cases} 1 & \text{if } a \geq 0, \\ -1 & \text{otherwise.} \end{cases}$

  Here w is the weight vector (containing weights like $w_1, w_2, \ldots$ from our examples) and b is the bias term. The inner product $w^\top x$ computes exactly the weighted sum we discussed earlier.
- Loss function $\ell(\hat{y}, y) = \begin{cases} 0 & \text{if } \hat{y} = y, \\ 1 & \text{otherwise.} \end{cases}$

  This 0/1 loss gives a penalty of 1 for each incorrect prediction, matching our intuitive notion of counting mistakes.

Before proceeding with the analysis, we make three simplifying assumptions that preserve the expressivity of our model while simplifying the theoretical analysis:

Remark 1 (Absorbing Bias). The bias term b in the decision function $\text{sgn}(w^\top x + b)$ can be absorbed into the weight vector by augmenting the input space. We have,

$$w^\top x + b = \begin{bmatrix} w_1 & w_2 & \cdots & w_d \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_d \end{bmatrix} + b = \begin{bmatrix} w_1 & w_2 & \cdots & w_d & b \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_d \\ 1 \end{bmatrix} = \tilde{w}^\top \tilde{x},$$

for $\tilde{x} := \begin{bmatrix} x \\ 1 \end{bmatrix} \in \mathbb{R}^{d+1}$ and $\tilde{w} := \begin{bmatrix} w \\ b \end{bmatrix} \in \mathbb{R}^{d+1}$ This augmentation increases the dimension by 1 but allows us to treat the bias as a standard weight parameter, simplifying our notation and analysis. For notational simplicity, after this augmentation we will continue to write x and w, with the understanding that they may now live in $\mathbb{R}^{d+1}$.

Remark 2 (Weight Normalization). The decision function $\text{sgn}(w^\top x)$ is invariant to positive scaling of w, since $\text{sgn}(\alpha w^\top x) = \text{sgn}(w^\top x)$ for any $\alpha > 0$. This scale invariance allows us to assume without loss of generality that $\|w\| = 1$ (and if we are using the augmented representation from Remark 1, this normalization applies to the full vector $\tilde{w} = [w; b]$). This normalization simplifies our geometric arguments and convergence analysis.

Remark 3 (Feature Scaling). We can assume $\|x_i\| \leq 1$ for all input vectors by appropriate scaling. For any dataset, let $\beta = \max_i \|x_i\|$ and replace each input with $x_i' := x_i/\beta$, so that $\|x_i'\| \leq 1$. This scaling is compatible with Remark 2: for any w, we have $\text{sgn}(w^\top x_i) = \text{sgn}((\beta w)^\top x_i')$, and scaling w by a positive constant does not change the classifier.

3 The Perceptron Algorithm

For now, we will make a *separability* assumption. In particular, we will assume that there exists a hyperplane that correctly classifies the data. Specifically, we assume there exists w_* such that for all $(x_i, y_i) \in S$, $y_i = \text{sgn}(w_*^\top x_i)$. We say the data is **linearly separable** if it satisfies this condition.

[1]We use $\{-1, 1\}$ instead of $\{0, 1\}$ for technical reasons that will become clear when analyzing the algorithm.

Algorithm 1: Perceptron

Initialize $w_1 = 0 \in \mathbb{R}^d$
for $t = 1, 2, \ldots$ **do**
  if $\exists i \in \{1, \ldots, m\}$ *s.t.* $y_i \neq \text{sgn}\left(w_t^\top x_i\right)$ **then** update $w_{t+1} = w_t + y_i x_i$
  else output w_t
end

The update may seem strange at first, but let us see the intuition behind it. Suppose x_i is the example that is misclassified and assume $\|x_i\|_2 = 1$:

- If the true label was positive ($y_i = +1$), then $w_{t+1}^\top x_i = (w_t + y_i x_i)^\top x_i = w_t^\top x_i + \|x_i\|^2 = w_t^\top x_i + 1$. Geometrically, adding $y_i x_i = x_i$ moves w_t towards the point, rotating the hyperplane to correctly classify it.
- If the true label was negative ($y_i = -1$), then $w_{t+1}^\top x_i = (w_t + y_i x_i)^\top x_i = w_t^\top x_i - \|x_i\|^2 = w_t^\top x_i - 1$. Geometrically, adding $y_i x_i = -x_i$ moves w_t away from the point, again rotating the hyperplane to correctly classify it.

In both cases, the update moves the weight in the right direction, increasing $y_i(w^\top x_i)$ by 1. Let's visualize this update geometrically:

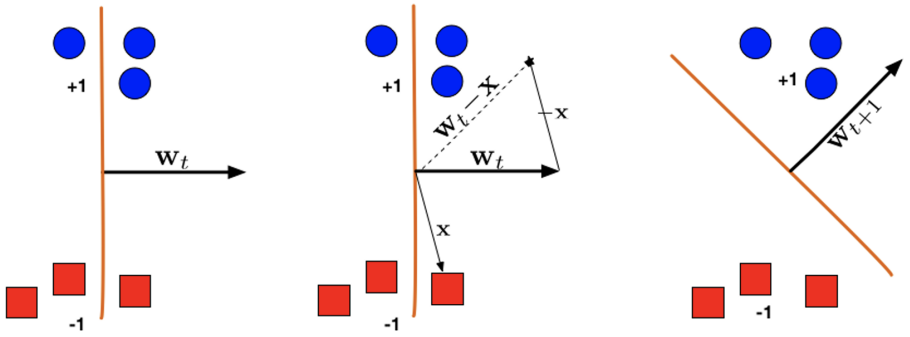


Figure 3: Perceptron update: the weight vector w_t rotates to correctly classify a misclassified point.

4 Theoretical Properties

Now that we understand how the Perceptron algorithm works, let's analyze its theoretical properties. A key question is: how many updates will the algorithm make before finding a solution? Looking at the update rule $w_{t+1} = w_t + y_i x_i$, we can see that each misclassified point causes the weight vector to change. But how many such changes will we need?

To answer this, let's think about what makes a point "easy" or "hard" for the Perceptron to classify correctly. When a point is far from the decision boundary, a small change in w is unlikely to flip its classification. However, points very close to the boundary are more sensitive—even a slight

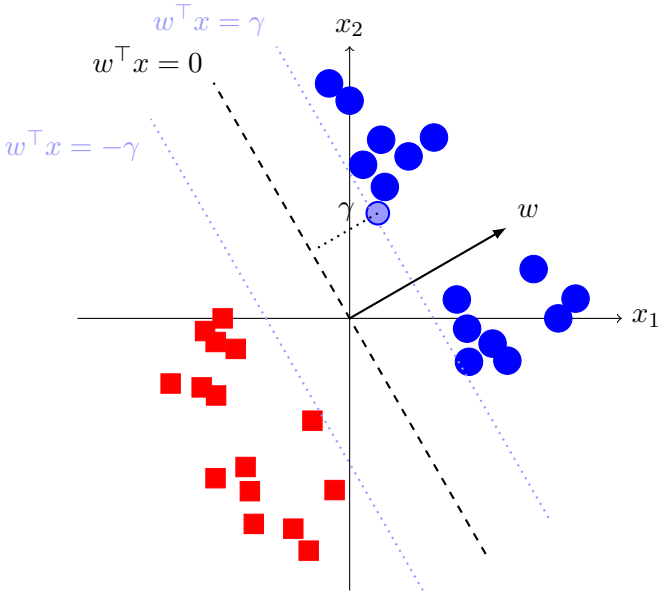


Figure 4: The margin γ is the minimum distance from any training point to the separating hyperplane.

rotation of w could change their predicted label. This suggests that the distance of points from the decision boundary is crucial for the Perceptron's behavior.

This intuition leads us to the concept of margin. A fundamental concept in our analysis is the margin γ, defined as the minimum distance of any training point from the separating hyperplane. For a unit vector w, this distance is given by $|w^\top x|$. Recall, w_* is the unit vector defining our target separating hyperplane (which exists by our separability assumption). Formally:

$$\gamma = \min_{i \in \{1,\ldots,m\}} |w_*^\top x_i|$$

Aside: Why $|w^\top x|$ gives the distance to the hyperplane

Consider any point x and let p be its projection onto the hyperplane $w^\top x = 0$. Since p lies on the hyperplane, $w^\top p = 0$. The vector from p to x must be parallel to w (perpendicular to hyperplane), so $x - p = kw$ for some scalar k. Taking the dot product with w (recall $\|w\| = 1$):

$$w^\top x = w^\top p + w^\top (kw) = 0 + k\|w\|^2 = k$$

Thus, $k = w^\top x$ is the signed distance, and $|w^\top x|$ is the absolute distance.

The margin plays a crucial role in the algorithm's convergence. We can prove that the Perceptron algorithm stops after at most $1/\gamma^2$ rounds. Let's understand this result step by step.

Main Idea. The proof follows an elegant strategy: 1. Show that each update brings w_t closer to the optimal solution w_* (progress) 2. Show that w_t can't grow too quickly in norm (control) 3.

Combine these to bound the number of updates

To formalize this intuition, we'll need two key properties:

Lemma 1 (Optimal Margin Lower Bound)**.** *The optimal separator w_* achieves margin at least γ on all points. That is, for all points i:*

$$y_i(w_*^\top x_i) \geq \gamma$$

Proof. Since w_* correctly classifies all points, $y_i = \operatorname{sgn}(w_*^\top x_i)$. Therefore:

$$y_i(w_*^\top x_i) = |w_*^\top x_i| \geq \min_j |w_*^\top x_j| = \gamma$$

where the first equality uses the fact that $\operatorname{sgn}(a)a = |a|$. □

Lemma 2 (Negative Prediction on Mistakes)**.** *For any point i that is misclassified by the current predictor w_t (meaning $y_i \neq \operatorname{sgn}(w_t^\top x_i)$), we have:*

$$y_i(w_t^\top x_i) \leq 0$$

Proof. If i is misclassified, then $y_i = -\operatorname{sgn}(w_t^\top x_i)$, so:

$$y_i(w_t^\top x_i) = -|w_t^\top x_i| \leq 0$$

□

Now we can prove our main convergence result:

Theorem 3. *The Perceptron algorithm stops after at most $1/\gamma^2$ updates, returning a hyperplane that correctly classifies all points.*

Proof. The key insight is that as w_t converges to the correct solution, it should align more closely with w_*. We can measure this alignment using the cosine similarity $\cos\theta_t = \frac{w_*^\top w_t}{\|w_t\|}$, which equals 1 when the vectors point in the same direction. For this angle to improve, we need the numerator to grow faster than the denominator.

Step 1: Progress. When we update on a misclassified point i:

$$\begin{aligned} w_*^\top w_{t+1} &= w_*^\top (w_t + y_i x_i) \\ &= w_*^\top w_t + y_i(w_*^\top x_i) \\ &\geq w_*^\top w_t + \gamma \qquad \text{(by Lemma ??)} \end{aligned}$$

The last step uses Lemma **??** which guarantees $y_i(w_*^\top x_i) \geq \gamma$ for any point.

Step 2: Control Growth. For the angle to improve, we also need to control how quickly $\|w_t\|$ grows:

$$\begin{aligned}\|w_{t+1}\|^2 &= \|w_t + y_i x_i\|^2 \\ &= \|w_t\|^2 + \underbrace{\|x_i\|^2}_{\leq 1} + 2\underbrace{y_i w_t^\top x_i}_{\leq 0} \\ &\leq \|w_t\|^2 + 1 \qquad \text{(by Lemma ??)}\end{aligned}$$

The key term is $y_i w_t^\top x_i$ which is negative for misclassified points by our Control Lemma.

Step 3: Combine. After T updates, we can bound both terms:

- For the numerator, applying Step 1 recursively:

$$\begin{aligned}w_*^\top w_{T+1} &\geq w_*^\top w_T + \gamma \\ &\geq (w_*^\top w_{T-1} + \gamma) + \gamma \\ &= w_*^\top w_{T-1} + 2\gamma \\ &\vdots \\ &\geq w_*^\top w_1 + T\gamma \\ &= T\gamma \quad (\text{since } w_1 = 0)\end{aligned}$$

- For the denominator, applying Step 2 recursively:

$$\begin{aligned}\|w_{t+1}\|^2 &\leq \|w_t\|^2 + 1 \\ &\leq \|w_{t-1}\|^2 + 2 \\ &\vdots \\ &\leq T \implies \|w_{T+1}\| \leq \sqrt{T}\end{aligned}$$

Therefore the cosine grows as:

$$\cos\theta_{T+1} = \frac{w_*^\top w_{T+1}}{\|w_{T+1}\|} \geq \frac{T\gamma}{\sqrt{T}} = \gamma\sqrt{T}$$

Since cosine is at most 1, we must have $\gamma\sqrt{T} \leq 1$, implying $T \leq 1/\gamma^2$. □

Interpretation. This bound reveals several key insights about the Perceptron algorithm. The convergence speed is directly controlled by the margin γ - when data points are well-separated (large γ), the algorithm converges quickly, but when points lie close to the decision boundary (small γ), convergence can be slow. Notably, the bound is independent of the input dimension d, making the Perceptron particularly efficient for high-dimensional problems where many other algorithms might struggle.

Implementation. Here's a simple implementation of the Perceptron algorithm in an IPython notebook that you can run yourself: Perceptron Implementation.

5 Limitations of Perceptron

Minsky and Papert wrote a book titled 'Perceptrons' in 1969 which showed that the Perceptron algorithm could not learn XOR functions (Figure **??**). Such data is not linearly separable. The book became widely popular as a criticism for neural networks (or perceptrons) and is said to have led to the AI winter which lasted till the mid-90s.

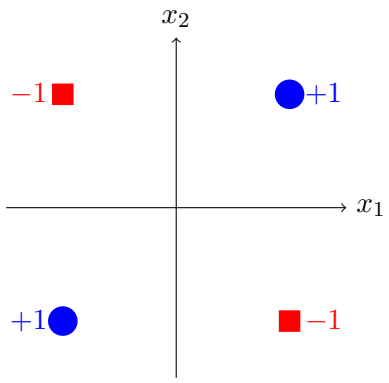


Figure 5: The XOR problem: no linear separator can correctly classify all points.

This limitation directly connects to our theoretical analysis: for the XOR problem, no positive margin γ exists (as no separating hyperplane exists at all), which explains why Perceptron cannot converge in this case. Apart from not working on non-linearly separable data, it cannot handle noise in the label. This has led to developments of other techniques that we will study in the later part of the course.